ASSIGNMENT - 9.3

Name: Y.sahasra

RollNo:2303A51269

B19

Task 1: Basic Docstring GenerationScenario

You are developing a utility function that processes numerical lists and must be properly documented for future maintenance.

Requirements

• Write a Python function to return the sum of even numbers and sum of odd numbers in a given list

• Manually add a Google Style docstring to the function

• Use an AI-assisted tool (Copilot / Cursor AI) to generate a function-level docstring

• Compare the AI-generated docstring with the manually written docstring

• Analyze clarity, correctness, and completeness

Expected Output

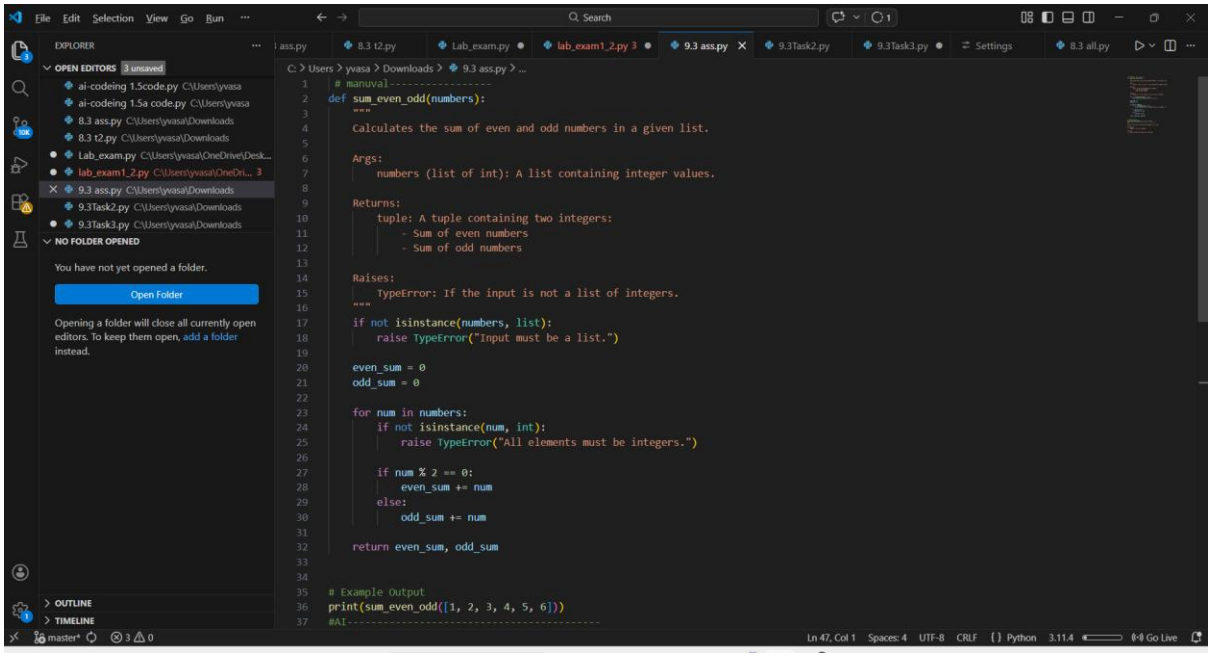
• Python function with manual Google-style docstring

• AI-generated docstring for the same function

• Comparison explaining differences between manual and AI-generated documentation

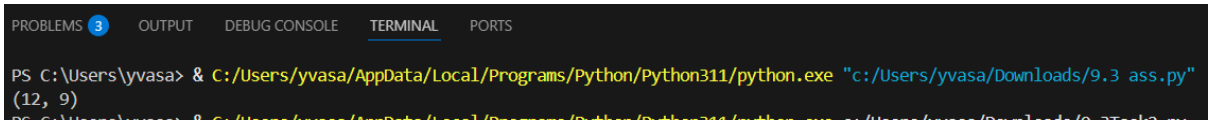
• Improved understanding of AI-generated function-level documentation

CODE:



```
# manual----------------
def sum_even_odd(numbers):
    """
    Calculates the sum of even and odd numbers in a given list.

    Args:
        numbers (list of int): A list containing integer values.

    Returns:
        tuple: A tuple containing two integers:
            - Sum of even numbers
            - Sum of odd numbers

    Raises:
        TypeError: If the input is not a list of integers.
    """
    if not isinstance(numbers, list):
        raise TypeError("Input must be a list.")

    even_sum = 0
    odd_sum = 0

    for num in numbers:
        if not isinstance(num, int):
            raise TypeError("All elements must be integers.")

        if num % 2 == 0:
            even_sum += num
        else:
            odd_sum += num

    return even_sum, odd_sum


# Example Output
print(sum_even_odd([1, 2, 3, 4, 5, 6]))
#AI----------------------------------------
```

OUTPUT:



```
PROBLEMS 3  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\yvasa> & C:/Users/yvasa/AppData/Local/Programs/Python/Python311/python.exe "c:/Users/yvasa/Downloads/9.3 ass.py"
(12, 9)
```

Task 2: Automatic Inline Comments

Scenario

You are developing a student management module that must be easy to understand for new developers.

Requirements

• Write a Python program for an sru_student class with the following:

– Attributes: name, roll_no, hostel_status

– Methods: fee_update() and display_details()

• Manually write inline comments for each line or logical block

• Use an AI-assisted tool to automatically add inline comments

• Compare manual comments with AI-generated comments

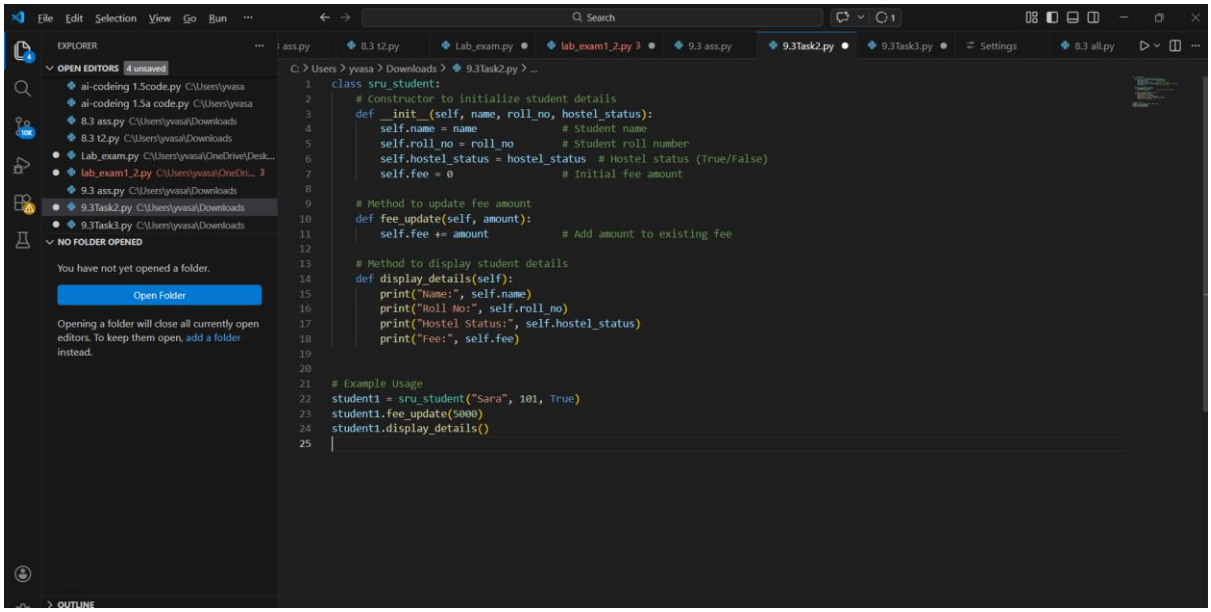
• Identify missing, redundant, or incorrect AI comments

Expected Output

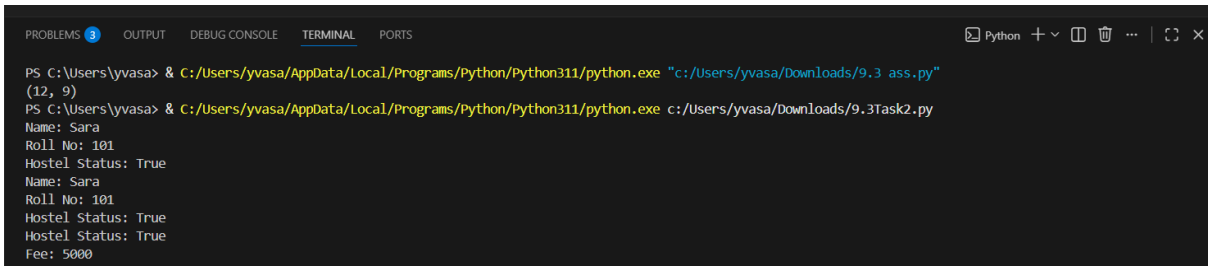
• Python class with manually written inline comments

- AI-generated inline comments added to the same code
- Comparative analysis of manual vs AI comments
- Critical discussion on strengths and limitations of AI-generated comments

CODE:



OUTPUT:



Task3: Module-Level and Function-Level Documentation

Scenario

You are building a small calculator module that will be shared across multiple projects and requires structured documentation.

Requirements

• Write a Python script containing 3–4 functions (e.g., add, subtract, multiply, divide)

• Manually write NumPy Style docstrings for each function

• Use AI assistance to generate:

– A module-level docstring

– Individual function-level docstrings

• Compare AI-generated docstrings with manually written ones

• Evaluate documentation structure, accuracy, and readability

Expected Output

• Python script with manual NumPy-style docstrings

• AI-generated module-level and function-level documentation

• Comparison between AI-generated and manual documentation

• Clear understanding of structured documentation for multi-function scripts

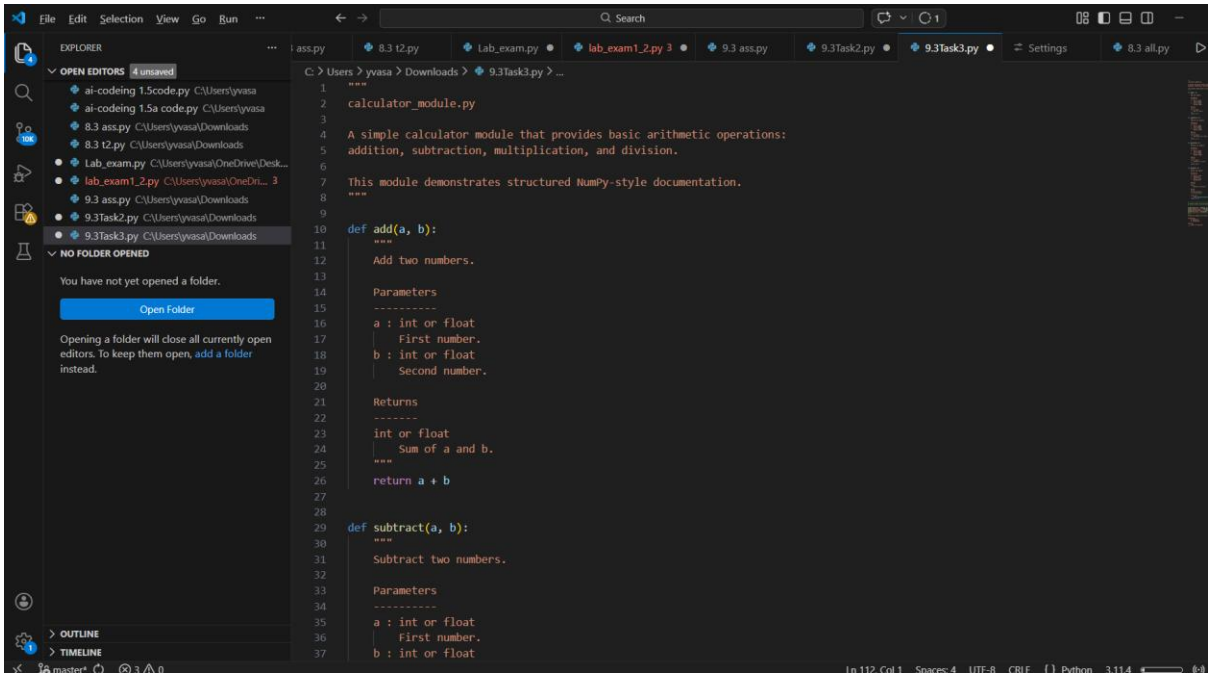
Additional Requirement

• Push the complete project documentation as a .md file to a GitHub repository

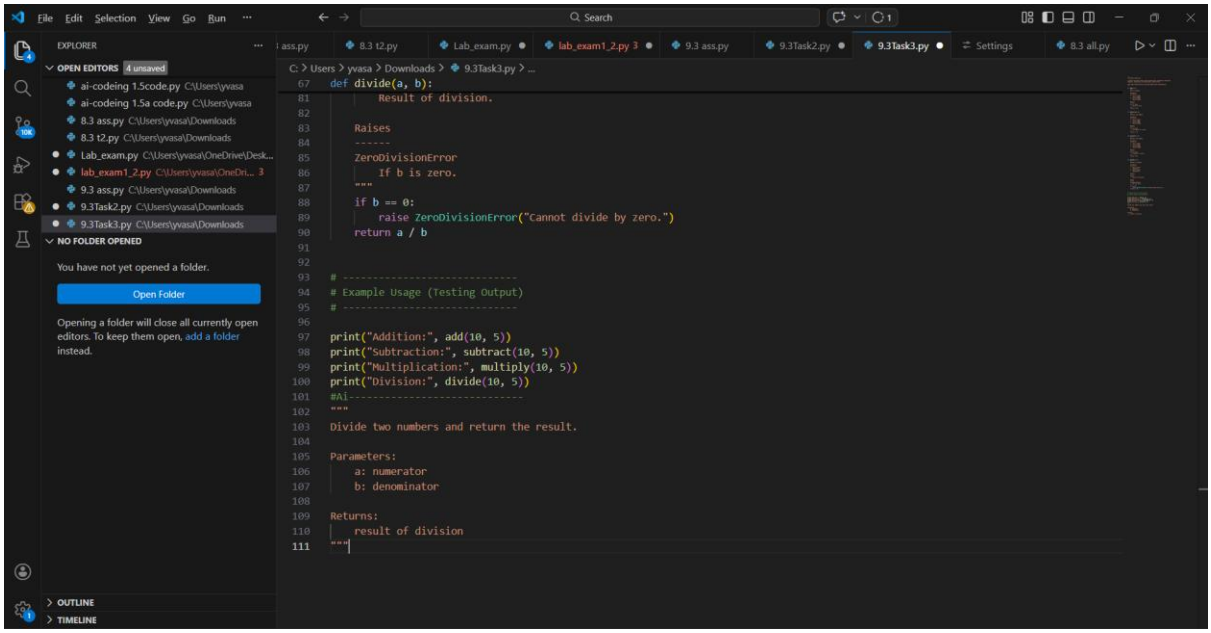
• Ensure documentation covers module overview and function descriptions

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

CODE:



```
"""
calculator_module.py

A simple calculator module that provides basic arithmetic operations:
addition, subtraction, multiplication, and division.

This module demonstrates structured NumPy-style documentation.
"""

def add(a, b):
    """
    Add two numbers.

    Parameters
    ----------
    a : int or float
        First number.
    b : int or float
        Second number.

    Returns
    -------
    int or float
        Sum of a and b.
    """
    return a + b


def subtract(a, b):
    """
    Subtract two numbers.

    Parameters
    ----------
    a : int or float
        First number.
    b : int or float
```



```
def divide(a, b):
        Result of division.

    Raises
    ------
    ZeroDivisionError
        If b is zero.
    """
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero.")
    return a / b


# ----------------------------
# Example Usage (Testing Output)
# ----------------------------

print("Addition:", add(10, 5))
print("Subtraction:", subtract(10, 5))
print("Multiplication:", multiply(10, 5))
print("Division:", divide(10, 5))
#Ai----------------------------
"""
Divide two numbers and return the result.

Parameters:
    a: numerator
    b: denominator

Returns:
    result of division
"""
```

OUTPUT:

```
PS C:\Users\yvasa> & C:/Users/yvasa/AppData/Local/Programs/Python/Python311/python.exe c:/Users/yvasa/Downloads/9.3Task3.py
Addition: 15
Addition: 15
Subtraction: 5
Multiplication: 50
Multiplication: 50
Division: 2.0
PS C:\Users\yvasa>
```